

Appunti su Basi di Dati

Riccardo Di Michele

1 introduzione

Nell'ambito informatico abbiamo sviluppato un bisogno di organizzare le *informazioni* che ci interessano, per poterle semplicemente conservare o *lavorarci*. Il modo in cui queste informazioni vengono rappresentate è tramite i **dati**, quest'ultimi devono però seguire una certa **codifica**; vediamo un esempio.



I dati che noi estraiamo da questi cartelli (le nostre informazioni) sono gli stessi: " *divieto dalle 8 alle 17* ". Ma in questo caso la codifica dei dati è molto importante poichè il modo in cui l'orario viene scritto ci darà *altretanti* dati. Infatti l'orario all'interno delle parentesi indica che riguarda solo il sabato, quello indicato in rosso è solo nei giorni festivi, senza nessun'altra formattazione invece tutti gli altri giorni.

Per ogni dato dovremmo quindi *specificare* come viene codificato, o semplicemente, come verrà interpretato, questo può essere un lavoro molto faticoso quando andremo a lavorare su migliaia e milioni di dati. Queste **basi di dati** verranno allora gestite da un **DBMS**, **DataBase Management System**. Un **DBMS** ci permette di gestire con facilità dati di **grandi** dimensioni (anche 500 Terabyte) e permette di condividere quest'ultimi attraverso più utenti (come tra i vari settori di una azienda); quest'ultima qualità potrebbe

però portare a qualche **conflitto**, se per esempio dipendente A vuole accedere allo stesso dato su cui sta lavorando dipendente B potrebbero formarsi dei dati non accurati.

Per questo motivo i **DBMS** offrono anche meccanismi di autorizzazione, quindi è possibile definire quale utente è autorizzato a leggere o modificare una certa raccolta di dati. Un aspetto delicato che deve gestire un **DBMS** è quindi quello delle **transizioni**, ovvero tutte quelle operazioni sui dati che devono essere **atomiche**, ovvero operazioni non scindibili, devono essere portate a termine per forza di cose, anche nel caso di transizioni concorrenti (come spiegato prima, quando due utenti operano sugli stessi dati).

2 Descrizione dei dati

Il **modello dei dati** è l'insieme delle tecniche e dei costrutti usati per organizzare i dati e descriverne la loro natura.

Per esempio il **modello relazionale** ci permette di costruire **record** omogenei; vediamo un esempio.

Orario

Insegnamento	Docente	Aula
Analisi I	Luigi Neri	N1
Chimica	Piero Rossi	N2
Basi di Dati	Angela Valenti	N3
Fisica II	Vittorio Strano	N4

La *relazione* di cui parla il modello non è altro che l'intera tabella Orario, ed è composta da righe omogenee, ovvero righe che hanno la stessa struttura: un **insegnamento**, un **docente** e un **aula**.

Sempre considerando la tabella possiamo adesso dare la definizione di **tupla** o (**record**) e **attributo**: la prima non è altro che una riga della tabella e la seconda è invece una colonna.

In ogni base di dati possiamo individuare uno **schema**, ovvero una *descrizione* invariante nel tempo che descrive la struttura della relazione; nella relazione *Orario* vista prima lo schema è l'insieme di tutti gli attributi (ovvero le colonne della relazione).

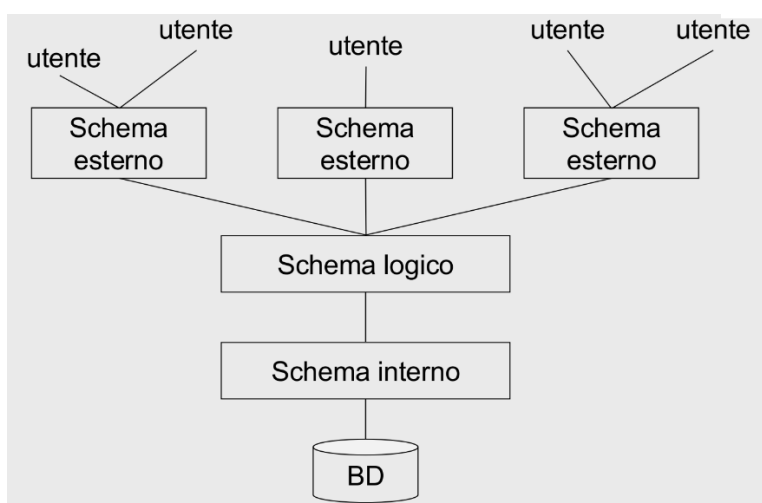
Troveremo inoltre l'**istanza**, che è invece variabile nel tempo, ovvero i valori della tabella stessa; la colonna *Insegnamento* è per esempio una istanza.

Abbiamo già parlato del *modello relazionale*, ma questo tipo fa in realtà

parte di una macro-categoria di modelli chiamati **modelli logici**, che descrivono i dati, e le relazioni tra loro, usando strutture "comprensibili" ad un software (per esempio tramite **XML**).

Un'altra categoria è quella dei **modelli concettuali** che invece mirano soltanto a rappresentare i dati a livello grafico, usando per esempio il famoso **schema E-R**, *Entità-Relazione*.

Di seguito vedremo una architettura molto comune per i DBMS, chiamata architettura **ANSI/SPARC**:



Ogni schema dell'architettura ha una sua funzione e soprattutto lavora soltanto su una parte del database.

Lo **schema esterno** è quello a cui si interfaccierà l'utente, di conseguenza sarà molto "semplificato". Se per esempio prendiamo il sito di una università l'utente medio sarà uno studente, lo schema esterno lavorerà quindi (e farà vedere) soltanto sui voti di quello studente; il database dell'università ha ovviamente altri dati (nomi dei professori, eventi, gli esami degli scorsi anni), ma farà solo vedere allo studente ciò che a lui davvero serve.

Al cuore della struttura troviamo invece lo **schema logico** che descrive in modo concettuale quali sono i dati e le relazioni tra quest'ultimi (per esempio tramite il *modello ER*, come detto prima)

L'ultimo è lo **schema interno** che descrive come sono *fisicamente* memorizzati questi dati e su quali dispositivi (*HDD*, *SDD* etc)

Il punto forte di questa struttura è l'**indipendenza dei dati**, infatti potremmo cambiare il modo in cui "descriviamo i dati" (schema logico), senza cambiare poi gli altri schemi o addirittura il database stesso.

3 Interrogare il database

Abbiamo parlato di come descriviamo e come conserviamo questi famigerati *dati*, ma come lavoriamo su quest'ultimi?

Useremo un linguaggio interattivo: **SQL**. Quest'ultimo è un linguaggio standard oramai per interfacciarsi con database e lo troveremo già incluso in tanti linguaggi come **C** oppure **Pascal**.

Il linguaggio è composto principalmente da due tipologie di operazioni: **DML** e **DDL**.

Le operazioni di **Data Manipulation Language** manipolano le istanze del database, e quindi modificano i dati al suo interno; il secondo tipo di operazioni, **Data Definition Language**, definiscono lo schema del database e come i dati si relazionino tra di loro.

Seguono degli esempi di, rispettivamente, operazione di DDL e DML

```
CREATE TABLE orario (  
    insegnamento CHAR(20),  
    docente CHAR(20),  
    aula CHAR(4),  
    ora CHAR(5)  
);  
  
INSERT INTO orario (insegnamento, docente, aula, ora)  
VALUES ('Basi di dati', 'Mario Rossi', 'N1', '09:00');
```

Questi esempi servono soltanto per avere un rapido chiarimento sulle differenze tra i due tipi di comandi, ovviamente una persona che non ha mai usato SQL avrà difficoltà a capire. Basta sapere che il primo comando **CREATE TABLE** definisce lo schema della tabella *orario*, il secondo comando (**INSERT INTO**), va invece a modificare questa tabella aggiungendo dei nuovi dati.

4 Personaggi del database

Infine vedremo le principali figure che si intersecano e lavorano col database. Il primo tra tutti è il **Database Administrator (DBA)**, che definisce lo schema della base di dati, gestisce la sicurezza del database (quindi decide chi è autorizzato ad accedere a cosa), si occupa della manutenzione, delle prestazioni e dei backup del database e, infine, coordina e gestisce le esigenze

dei vari utenti; per esempio se una azienda A vuole accedere agli stessi dati di una azienda B il **DBA** progetterà la base di dati in modo tale che entrambe le aziende siano soddisfatte. Abbiamo inoltre la figura del **programmatore**, che si occupa di scrivere applicazioni che interfaccino l'utente col database, in base al tipo di utente ci saranno ovviamente applicativi diversi; per esempio un utente comune non avrà sicuramente accesso a ogni dato del database di una banca.

In modo simile gli **analisti** invece sono utenti ma con conoscenze del linguaggio SQL, quindi non usano applicativi ma sono loro stessi che "interrogano" il database con query scritte da loro.

Infine gli **utenti finali** (detti **naviganti**) sono gli utenti più comuni, quelli che interagiscono con le applicazioni scritte dai *programmatori* già citati prima.

5 Modello relazionale

Nel senso stretto del termine, la *relazione* di cui parla il modello è una relazione **matematica**; prendiamo la seguente tabella come esempio

a	1
a	2
b	1
b	2

Questa è la tabella nata dal prodotto cartesiano di due due insiemi, $D_1 = \{1, 2\}$ e $D_2 = \{a, b\}$ e la relazione r sarà quindi solo parte di queste combinazioni, quindi $r \subseteq D_1 \times D_2$; per esempio imponiamo che la nostra relazione sia "*il numero corrispondente alla lettera*" :

a	1
b	2

 $\subseteq$

a	1
a	2
b	1
b	2

Proprio come nella matematica le due tabelle originali D_1 e D_2 saranno i **domini** della relazione.

Un elemento della relazione potrà quindi essere la d-upla $(a, 1)$ e possiamo notare come la **posizione** degli elementi all'interno della d-upla è molto importante, infatti sappiamo che a viene dalla prima tabella e 1 dalla seconda. La relazione sarà quindi un insieme di n-uple ordinate del tipo

$$(d_1, d_2 \dots d_n) \quad | \quad d_1 \in D_1, d_2 \in D_2 \dots d_n \in D_n$$

Molto più diffuse sono invece le strutture **non posizionali**, ovvero quelle dove la posizione degli elementi della enn-upla non ha importanza, questo perchè ogni elemento è invece appartenente ad un **attributo**.

Casa	Fuori	RetiCasa	RetiFuori
Juve	Lazio	3	1
Lazio	Milano	0	1
Juve	Roma	2	2
Roma	Milano	0	3

Come si vede in questo esempio ogni dominio ha desso un *attributo* che descrive il suo scopo.

Dobbiamo specificare che il modello è basato su **valori**, ogni dato al suo interno sono valori, non abbiamo matrici, array o addirittura puntatori, all'interno delle nostre tabelle. Capiterà però che avremo bisogno di fare **riferimento** a dati presenti in altre relazioni; voglio inoltre specificare che in futuro vedremo più nel dettaglio questo particolare caso ma per ora vediamo come viene "visualizzato" nelle tabelle.

studenti	Matricola	Cognome	Nome	Data di nascita
	6554	Rossi	Mario	05/12/1978
	8765	Neri	Paolo	03/11/1976
	9283	Verdi	Luisa	12/11/1979
	3456	Rossi	Maria	01/02/1978
esami	Studente	Voto	Corso	
	3456	30	04	
	3456	24	02	
	9283	28	01	
	6554	26	01	

Come vediamo il valore della colonna *Studente* nella relazione *esami* fa riferimento al valore all'interno della colonna *Matricola* nella relazione *studenti*.

Poichè il modello usa valori ci servirà anche un **tipo** di valore per rappresentare valori sconosciuti, inesistenti o senza informazione: useremo quindi

il valore **NULL**. Un altro caso molto comune è quello dove invece conosciamo il valore ma deve rispettare dei **vincoli**, prendiamo come esempio la relazione *esami* vista prima.

Il voto universitario deve essere un intero compreso tra 18 e 30, questo implica un **vincolo**. Infatti qualsiasi sia il modo di immettere dati all'interno del nostro database dovremo imporre un **vincolo d'integrità** e controllare che qualsiasi numero sia stato inserito rispetti questi vincoli.

Andando più nello specifico l'esempio appena fatto fa parte di una serie di vincoli chiamati **vincoli intra-relazionali**, ovvero quei vincoli che interessano una tabella.

L'esempio sul voto è un **vincolo di dominio**, poichè tutto il dominio dei voti deve rispettare quel vincolo, ma abbiamo anche il **vincolo di ennupla**, che invece interessano più valori della stessa riga; per esempio se abbiamo la relazione *stipendio*, che raccoglie lo stipendio dei lavoratori ogni mese e infine uno *stipendio annuale*, quest'ultima colonna dovrà essere esattamente la somma di ogni stipendio mensile.

Infine abbiamo un'altra categoria di vincoli, quelli **inter-razionali** che interessano invece più relazioni; in questo caso semplicemente il vincolo impone che se io faccio riferimento ad un dato di un'altra tabella quel dato deve esistere in quella tabella.

Identificare una riga all'interno di una tabella è molto importante, potrebbe servirci capire quale riga crea problemi al database o potrebbe essere utile semplicemente cancellarla. Per questo motivo esiste un altro *vincolo indiretto*, quello della **chiave**. Il vincolo impone infatti che ogni riga della tabella sia diversa dalle altre. Per essere tale basta che una ennupla sia diversa dalle altre, anche di un solo valore.

L'insieme di dati che identificano una ennupla prende il nome di **superchiave**.

Matricola	Nome	Cognome	Corso
12345	Walter	Saitta	Ing. Informatica
22234	Federico	Bellomia	Ing. Edile
90311	Veronica	Ciulla	Psicologia
04316	Luca	Tigre	Architettura

In una tabella di questo tipo possiamo subito usare come *superchiave* l'attributo **Matricola** poichè, per come è definita una matricola, non esisteranno due ennuple con matricole identiche; non solo, potremmo anche

usare l'insieme di attributi (*Matricola, Nome, Cognome*), con la stessa logica usata prima possiamo dire che non esisterà altro studente con lo stesso nome, cognome e matricola di un altro.

C'è però una grossa distinzione tra le due, la prima (*Matricola*) è infatti una **chiave**, ovvero il set di attributi **minimi** che identifica una riga della tabella; quindi ogni *chiave* è anche una *superchiave*, ma non è detto il contrario.

Un altro tipo di chiave è la **chiave primaria**, un tipo di chiave che non accetta valori nulli, per esempio un identificativo come la *matricola* non sarà mai nullo, quindi vale come chiave primaria.

Similmente al vincolo *chiave* parleremo del **vincolo d'integrità referenziale**, ovvero della **foreign key**, quest'ultima è un chiave (primaria o non) appartenente ad una tabella A che compare anche nella tabella B ma non come chiave. Di conseguenza se nella tabella A la chiave avrà valore X, quando poi la useremo come *foreign key* nella tabella B sarà vincolata ad avere il valore X.

<u>Codice</u>	Data	Vigile	Prov	Numero
34321	1/2/15	3987	MI	39548K
53524	4/3/15	3295	TO	E39548
64521	5/4/16	3295	PR	839548
73321	5/2/18	9345	PR	839548

Auto	<u>Prov</u>	<u>Numero</u>	Cognome	Nome
	MI	39548K	Rossi	Mario
	TO	E39548	Rossi	Mario
	PR	839548	Neri	Luca

Come vediamo i valori delle tabelle *Prov* e *Numero* della tabella sopra compaiono identiche nella tabella *Auto*.